

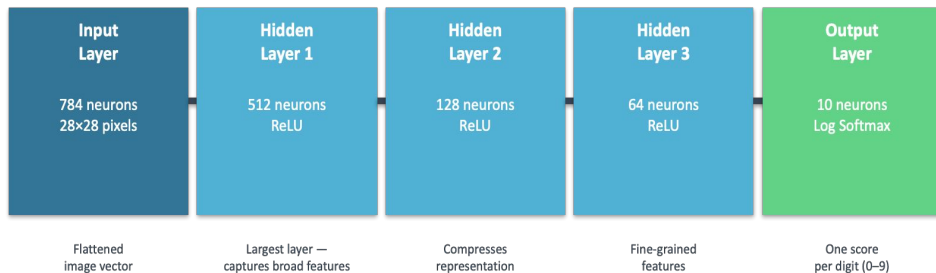
Group 7
8 May 2026

CSCI 580 MLP Final Project

Jesse Dalton, Christian Utrup, Josue Jauregui

Baseline – Architecture

```
# class structure for nn
class Network(nn.Module):
    # making of network
    def __init__(self):
        super().__init__() # try experimenting with different layer combinations for better results
        self.fc1 = nn.Linear(784,512)
        self.fc2 = nn.Linear(512,128)
        self.fc3 = nn.Linear(128,64)
        self.output = nn.Linear(64,10)
    # feedforward process
    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = F.relu(self.fc3(x))
        x = F.log_softmax(self.output(x), dim=1)
        return x
```



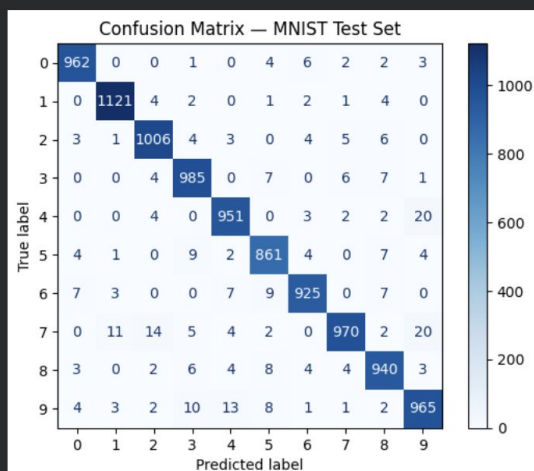
- Epochs 20
- Lr = 0.001
- batch = 64
- Relu activation function: relu()
- Loss function: NLLLoss()
- Optimizer: SGD()

```
# Define variables and model
epochs = 20
model = Network()
optimizer = torch.optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
loss_fn = nn.NLLLoss()
```

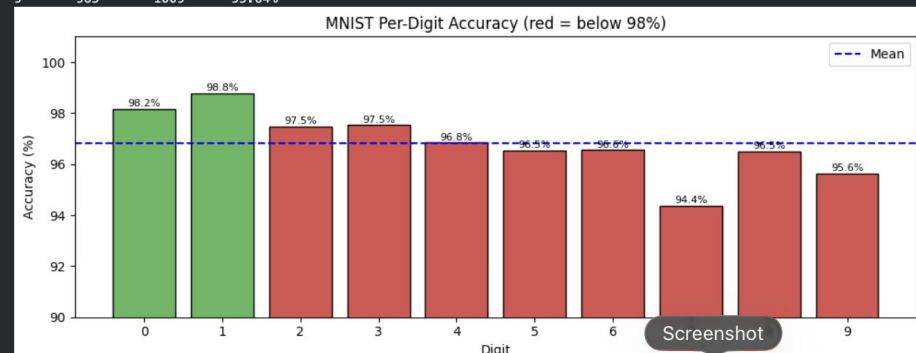
Baseline - MNIST Results

MNIST Test Accuracy: 96.86%
MNIST Error Rate: 3.14%

	precision	recall	f1-score	support
0	0.98	0.98	0.98	980
1	0.98	0.99	0.99	1135
2	0.97	0.97	0.97	1032
3	0.96	0.98	0.97	1010
4	0.97	0.97	0.97	982
5	0.96	0.97	0.96	892
6	0.97	0.97	0.97	958
7	0.98	0.94	0.96	1028
8	0.96	0.97	0.96	974
9	0.95	0.96	0.95	1009
accuracy			0.97	10000
macro avg	0.97	0.97	0.97	10000
weighted avg	0.97	0.97	0.97	10000



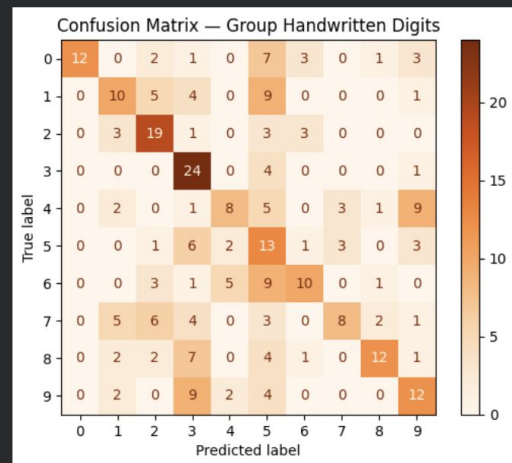
MNIST Per-Digit Accuracy:			
Digit	Correct	Total	Accuracy
0	962	980	98.16%
1	1121	1135	98.77%
2	1006	1032	97.48%
3	985	1010	97.52%
4	951	982	96.84%
5	861	892	96.52%
6	925	958	96.56%
7	970	1028	94.36%
8	940	974	96.51%
9	965	1009	95.64%



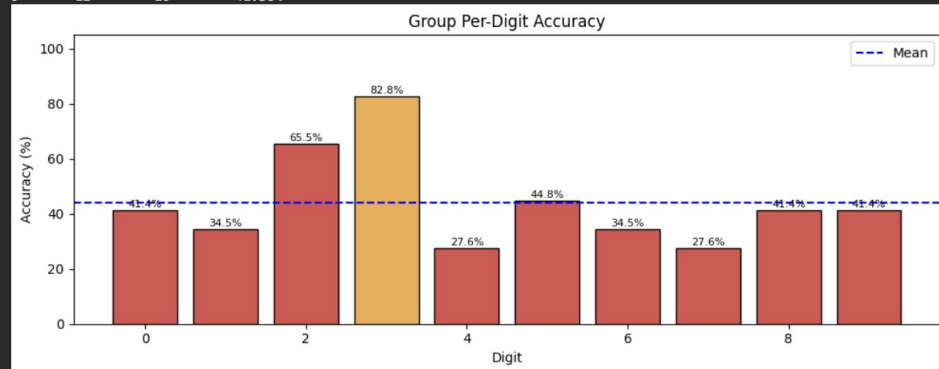
Baseline - Class Results

Group Digit Accuracy: 44.14%
Group Error Rate: 55.86%

	precision	recall	f1-score	support
0	1.00	0.41	0.59	29
1	0.42	0.34	0.38	29
2	0.50	0.66	0.57	29
3	0.41	0.83	0.55	29
4	0.47	0.28	0.35	29
5	0.21	0.45	0.29	29
6	0.56	0.34	0.43	29
7	0.57	0.28	0.37	29
8	0.71	0.41	0.52	29
9	0.39	0.41	0.40	29
accuracy			0.44	290
macro avg	0.52	0.44	0.44	290
weighted avg	0.52	0.44	0.44	290



Digit	Correct	Total	Accuracy
0	12	29	41.38%
1	10	29	34.48%
2	19	29	65.52%
3	24	29	82.76%
4	8	29	27.59%
5	13	29	44.83%
6	10	29	34.48%
7	8	29	27.59%
8	12	29	41.38%
9	12	29	41.38%



Testing/Design Strategy:

- **Set one independent variable and conduct tests with different values. Then pick the best outcome and repeat.**
- **Ranges Tested: Epochs: 10 → 50 LR: 0.0001 → 0.01 Batch: 32, 64, 128 Optimizer: SGD, Adam Loss: NLLLoss, CrossEntropy Layers: 2–4**

Loss function + Optimizer Testing (standardized at 20 epochs, lr = 0.001 and momentum = 0.9):

Test 1: SGD + Negative Log Likelihood Loss

MNIST Test Accuracy: ~96.86%
Group Digit Accuracy: ~44.14%

Test 2: SGD + Cross Entropy Loss

MNIST Test Accuracy: ~96.63%
Group Digit Accuracy: ~46.21%

Test 3: Adam + Negative Log Likelihood Loss

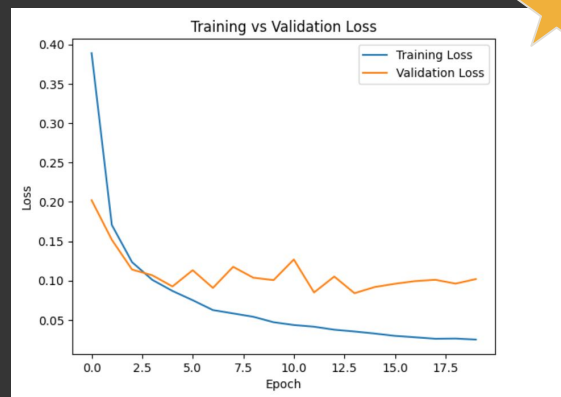
MNIST Test Accuracy: ~97.68%
Group Digit Accuracy: ~54.83%



Test 4: Adam + Cross Entropy Loss

MNIST Test Accuracy: 96.91%
Group Digit Accuracy: 51.38%

Architecture Testing (Prototyping):



```
self.fc1 = nn.Linear(784,128)
```

```
self.fc2 = nn.Linear(128,64)
```

```
self.output = nn.Linear(64,10)
```

MNIST Test Accuracy: ~97.16%

Group Digit Accuracy: ~54.83%

```
self.fc1 = nn.Linear(784,512)
```

```
self.fc2 = nn.Linear(512,128)
```

```
self.fc3 = nn.Linear(128,64)
```

```
self.output = nn.Linear(64,10)
```

MNIST Test Accuracy: ~97.70%

Group Digit Accuracy: ~59.31%

```
self.fc1 = nn.Linear(784,512)
```

```
self.fc2 = nn.Linear(512,256)
```

```
self.fc3 = nn.Linear(256,128)
```

```
self.fc4 = nn.Linear(128,64)
```

```
self.output = nn.Linear(64,10)
```

MNIST Test Accuracy: ~98.06%

Group Digit Accuracy: ~57.24%

Independent Variable = LR

Test 1:
Epochs = 30
Batch size = 64

LR = 0.0001

MNIST Test Accuracy:

~97.95%

Group Digit Accuracy:

~53.45%

Test 2:
Epochs = 30
Batch size = 64

LR = 0.0005

MNIST Test Accuracy:

~97.56%

Group Digit Accuracy:

~55.86%

Test 3:
Epochs = 30
Batch size = 64

LR = 0.001

MNIST Test Accuracy:

~97.89%

Group Digit Accuracy:

~57.93%



Test 4:
Epochs = 30
Batch size = 64

LR = 0.0015

MNIST Test Accuracy:

~97.65%

Group Digit Accuracy:

~56.90%

Test 5:
Epochs = 30
Batch size = 64

LR = 0.01

MNIST Test Accuracy:

~95.19%

Group Digit Accuracy:

~47.93%

Independent Variable = Batch Size

Test 1:
Epochs = 30
Batch size = 32

LR = 0.001

MNIST Test Accuracy:

- 97.36%

Group Digit Accuracy:

- 59.31%

Test 2:
Epochs = 30
Batch size = 64

LR = 0.001

MNIST Test Accuracy:

- 97.69%

Group Digit Accuracy:

- 55.53%

Test 3:
Epochs = 30
Batch size = 128

LR = 0.001

MNIST Test Accuracy:

- 97.80%

Group Digit Accuracy:

- 61.72%



Independent Variable = Epochs

Test 1:
Epochs = 10
Batch size = 128

LR = 0.001

MNIST Test Accuracy:

- 97.39%

Group Digit Accuracy:

- 56.21%

Test 2:
Epochs = 20
Batch size = 128

LR = 0.001

MNIST Test Accuracy:

- 97.73%

Group Digit Accuracy:

- 56.55%

Test 3:
Epochs = 30
Batch size = 128

LR = 0.001

MNIST Test Accuracy:

~ 97.87%

Group Digit Accuracy:

~ 61.38%



Test 4:
Epochs = 40
Batch size = 128

LR = 0.001

MNIST Test Accuracy:

~97.57%

Group Digit Accuracy:

~54.83%

Test 5:
Epochs = 50
Batch size = 128

LR = 0.001

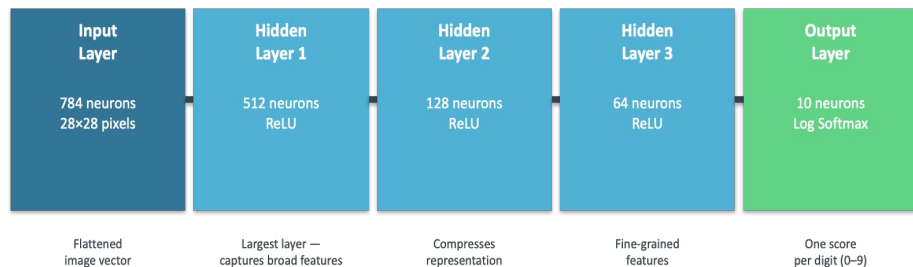
MNIST Test Accuracy:

~97.69%

Group Digit Accuracy:

~57.24%

Final Architecture/Setup:



Final Product:

- Relu activation function: `relu()`
- Loss function: `NLLLoss()`
- Optimizer: `Adam()`

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
loss_fn = nn.NLLLoss()
```

```
# MLP Network class: 784 -> 512 -> 128 -> 64 -> 10
# Uses ReLU activations for hidden layers and LogSoftmax for output
# LogSoftmax is used with NLLLoss as shown in class slides
# class structure for nn
class Network(nn.Module):
    # making of network
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784,512)
        self.fc2 = nn.Linear(512,128)
        self.fc3 = nn.Linear(128,64)
        self.output = nn.Linear(64,10)

    # feedforward process
    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = F.relu(self.fc3(x))

        x = F.log_softmax(self.output(x), dim=1)
        return x
```

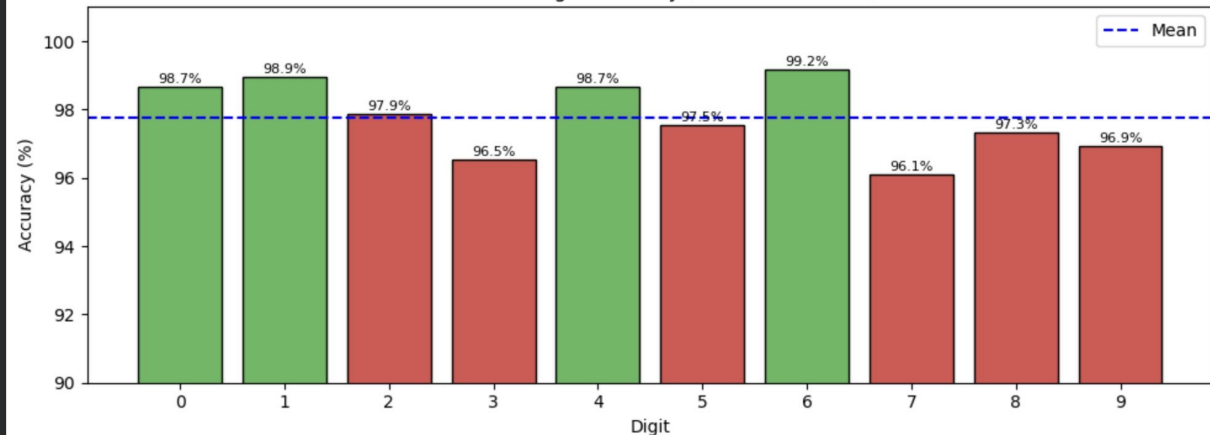

Analysis/ Results - MNIST

MNIST Per-Digit Accuracy:

Digit	Correct	Total	Accuracy
-------	---------	-------	----------

0	967	980	98.67%
1	1123	1135	98.94%
2	1010	1032	97.87%
3	975	1010	96.53%
4	969	982	98.68%
5	870	892	97.53%
6	950	958	99.16%
7	988	1028	96.11%
8	948	974	97.33%
9	978	1009	96.93%

MNIST Per-Digit Accuracy (red = below 98%)



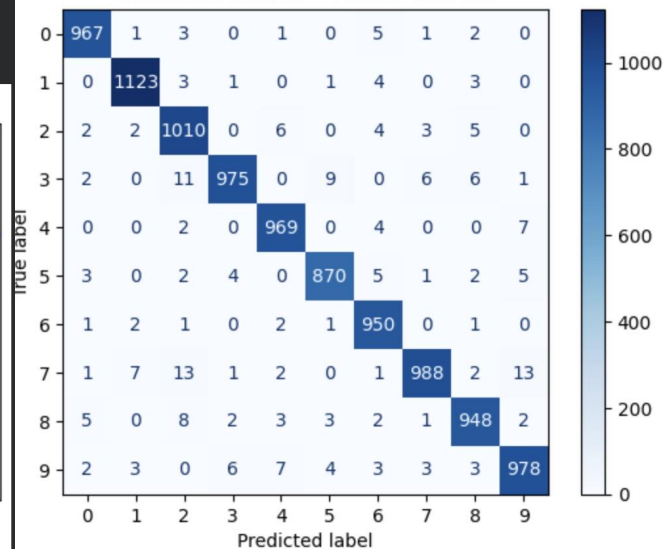
MNIST Test Accuracy: 97.78%

MNIST Error Rate: 2.22%

	precision	recall	f1-score	support
0	0.98	0.99	0.99	980
1	0.99	0.99	0.99	1135
2	0.96	0.98	0.97	1032
3	0.99	0.97	0.98	1010
4	0.98	0.99	0.98	982
5	0.98	0.98	0.98	892
6	0.97	0.99	0.98	958
7	0.99	0.96	0.97	1028
8	0.98	0.97	0.97	974
9	0.97	0.97	0.97	1009

accuracy			0.98	10000
macro avg	0.98	0.98	0.98	10000
weighted avg	0.98	0.98	0.98	10000

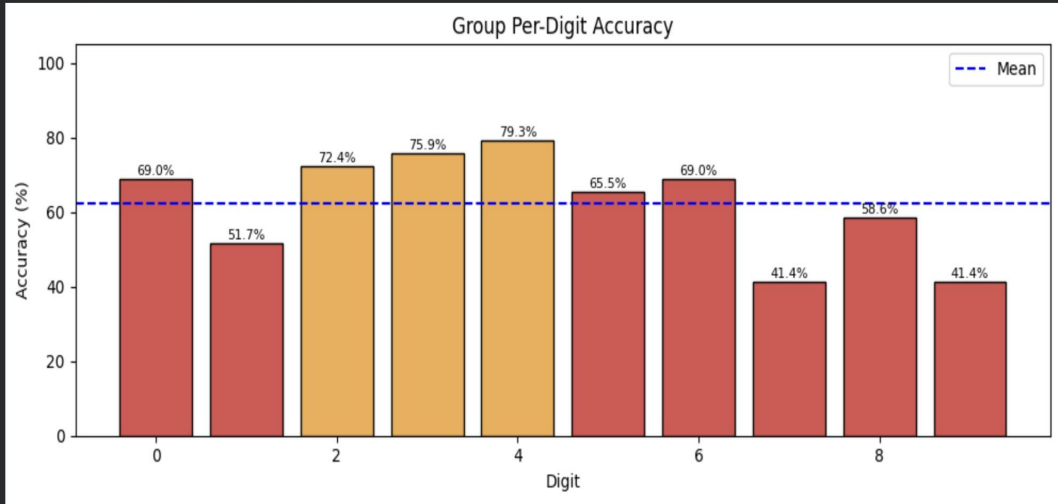
Confusion Matrix — MNIST Test Set



Analysis/ Results - Class Images

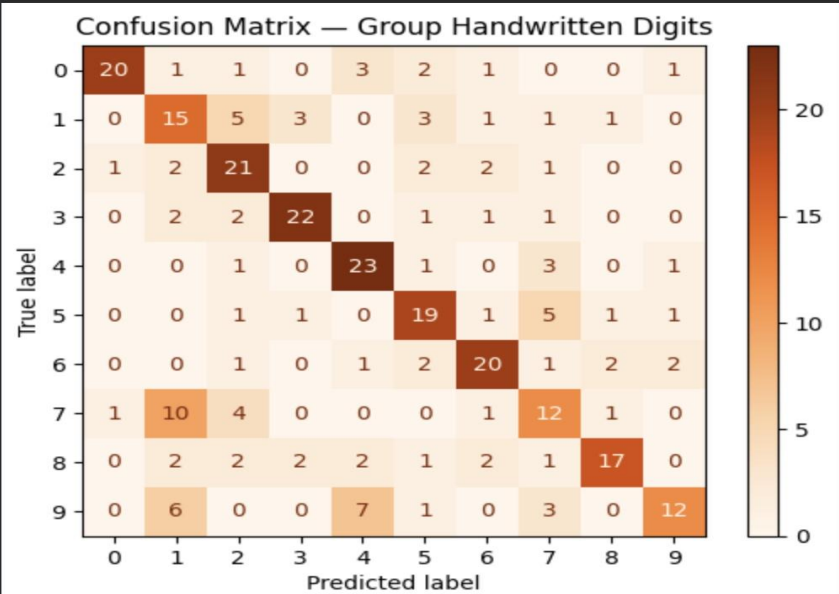
Group Per-Digit Accuracy:

Digit	Correct	Total	Accuracy
0	20	29	68.97%
1	15	29	51.72%
2	21	29	72.41%
3	22	29	75.86%
4	23	29	79.31%
5	19	29	65.52%
6	20	29	68.97%
7	12	29	41.38%
8	17	29	58.62%
9	12	29	41.38%



Group Digit Accuracy: 62.41%
Group Error Rate: 37.59%

	precision	recall	f1-score	support
0	0.91	0.69	0.78	29
1	0.39	0.52	0.45	29
2	0.55	0.72	0.63	29
3	0.79	0.76	0.77	29
4	0.64	0.79	0.71	29
5	0.59	0.66	0.62	29
6	0.69	0.69	0.69	29
7	0.43	0.41	0.42	29
8	0.77	0.59	0.67	29
9	0.71	0.41	0.52	29
accuracy			0.62	290
macro avg	0.65	0.62	0.63	290
weighted avg	0.65	0.62	0.63	290



Critical Code Walkthrough:

```
# Main loop – trains on MNIST dataset with validation after each epoch
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)
print(f"Training on: {device}")

for epoch in range(epochs):
    # Training phase
    model.train()
    running_train_loss = 0.0
    for images, labels in trainloader:
        images = images.view(images.shape[0], -1).to(device)
        labels = labels.to(device)
        optimizer.zero_grad()
        output = model(images)
        loss = loss_fn(output, labels) # NLLLoss
        loss.backward()
        optimizer.step()
        running_train_loss += loss.item()

    # Validation phase (after each epoch)
    model.eval()
    running_val_loss = 0.0
    with torch.no_grad():
        for images, labels in valloader:
            images = images.view(images.shape[0], -1).to(device)
            labels = labels.to(device)
            output = model(images)
            loss = loss_fn(output, labels)
            running_val_loss += loss.item()

    # Record and print epoch results
    avg_train_loss = running_train_loss / len(trainloader)
    avg_val_loss = running_val_loss / len(valloader)
    trainLoss.append(avg_train_loss)
    valLoss.append(avg_val_loss)
    print(f'Epoch {epoch+1}/{epochs} – Train Loss: {avg_train_loss:.4f} – Val Loss: {avg_val_loss:.4f}')

torch.save(model.state_dict(), 'best_model.pth')
print("\nTraining complete. Model saved.")
```

images.view() -> "Flatten 28×28 -> 784 vector"

optimizer.zero_grad() -> "Reset gradients each batch"

model(images) -> "Feedforward process"

Loss_fn -> nn.NLLLoss()

loss.backward() -> "Calculate gradients for each weight"

optimizer.step() -> "Update weights based on the computed gradients"

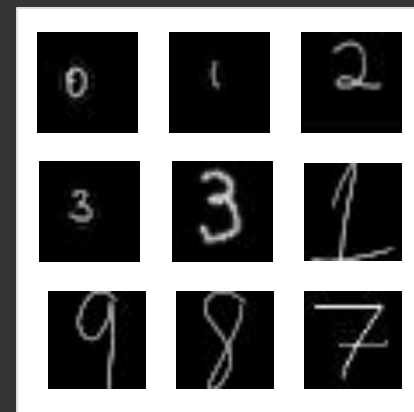
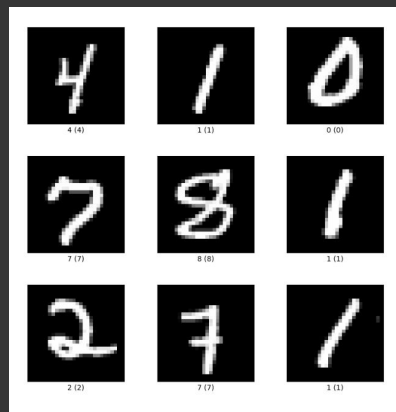
Why was the class accuracy so low?

Possibilities:

- Centering
- Sizing/Coverage
- Blurriness
- Thickness
- Contrast

We Covered:

- Resized Images
- Conversion to grayscale



MNIST Dataset vs Class Dataset

Group Per-Digit Accuracy:

Digit	Correct	Total	Accuracy
-------	---------	-------	----------

0	20	29	68.97%
1	15	29	51.72%
2	21	29	72.41%
3	22	29	75.86%
4	23	29	79.31%
5	19	29	65.52%
6	20	29	68.97%
7	12	29	41.38%
8	17	29	58.62%
9	12	29	41.38%

Team Group accuracies

Group	Correct	Total	Accuracy
1	5	20	25.00%
2	14	30	46.67%
3	15	40	37.50%
4	21	30	70.00%
5	24	30	80.00%
6	23	30	76.67%
7	23	30	76.67%
8	29	40	72.50%
9	27	40	67.50%